Notes_25.09: kNN - K Nearest Neighbours (Intuition & Distance Metrics)

Python 3.10+

Jupyter Notebook

PDF notes.pdf

🌐 **Language: English** | हिंदी / Hinglish | ⬅️ Lecture 25 Overview

📌 Executive Summary

Non-parametric lazy learner, Euclidean, Manhattan, and Minkowski distance metrics, choosing K.

🎯 Mathematical Formulations

Minkowski Distance Metric:

$$D_p(\mathbf{x}, \mathbf{z}) = \left(\sum_{j=1}^{d} |x_j - z_j|^p \right)^{1/p}$$

- For $p = 1$: **Manhattan Distance** (L_1 norm): $D_1 = \sum |x_j - z_j|$
- For $p = 2$: **Euclidean Distance** (L_2 norm): $D_2 = \sqrt{\sum (x_j - z_j)^2}$

💡 Conceptual Principles & Architecture

kNN Kaise Kaam Karta Hai? (Lazy Learner):

1. Training phase mein kNN koi formula ya weights train nahi karta; yeh poore training data ko memory mein store kar leta hai (isliye ise 'Instance-based' ya 'Lazy Learner' bolte hain).
2. Prediction ke waqt, naye query point $\mathbf{x}$ se training set ke saare points ka distance calculate hota hai.
3. Sabse paas wale K points (nearest neighbours) dhunde jaate hain.
4. Majority vote se winner class predict ho jati hai!

K Ki Value Ka Asar:

- **Small K (e.g. $K = 1$):** Model noise memorize karega, decision boundary bohot complex aur wiggly hogi $\implies$ **Overfitting (High Variance)**.
- **Large K (e.g. $K = 100$):** Model majority class predict karega, local patterns miss karega $\implies$ **Underfitting (High Bias)**.
- **Rule of Thumb:** $K = \sqrt{N}$ (aur binary classification mein tie break karne ke liye hamesha Odd number choose karein).

💻 Python Production Implementation

```
import numpy as np

p1 = np.array([1.0, 2.0])
p2 = np.array([4.0, 6.0])

l2_dist = np.linalg.norm(p1 - p2)  # Euclidean
l1_dist = np.sum(np.abs(p1 - p2))  # Manhattan

print(f"Euclidean Distance (L2): {l2_dist:.4f}")
print(f"Manhattan Distance (L1): {l1_dist:.4f}")
```